

1) Funktionale Abhängigkeiten – Empirisch

Note: natürlich gilt immer $\{x\} \twoheadrightarrow \{x\}$. Diese Beziehungen werden hier weggelassen.

```
{A} --> {D} // 1 bildet zwei mal auf 4
{C} --> {D} // 1 bildet zwei mal auf 4
{B} --> {A} // alles in B unterschiedlich -- kann auf alles abbilden
{B} --> {C} // B bildet auch auf D, aber D ist auch über A oder C erreichbar
{A, C} --> {B}
```

Weitere funktionale Abhängigkeiten sind nicht minimal. So ist z.B. $\{A, B\} \twoheadrightarrow \{C\}$ nicht minimal, da $\{B\} \twoheadrightarrow \{C\}$ gilt.

2) Funktionale Abhängigkeiten – Theoretisch

```
{STGKennzahl} --> {STGBezeichnung} // nicht unbedingt umgekehrt - z.B. könnte eine alte
und neue Version eines Studiengangs unter dem selben Namen gespeichert sein
{StudentID} --> {Vorname, Nachname, Herkunftsland}
{StudentID, STGKennzahl} --> {Semester}
```

// das sind die minimalen funktionalen Abhängigkeiten. Natürlich gilt auch z.B. $\{StudentID\} \twoheadrightarrow \{Vorname\}$ oder $\{StudentID, STGKennzahl\} \twoheadrightarrow \{Semester\}$, aber diese FA sind nicht minimal.

3.) Normalformen

Bauanleitung(Produkt, Teil, Schritt, Beschreibung, Menge)

$F = \{\{\text{Produkt, Teil, Schritt}\} \twoheadrightarrow \{\text{Menge}\}, \{\text{Produkt}\} \twoheadrightarrow \{\text{Beschreibung}\}\}$

a)

- Schlüssel: {Produkt, Teil, Schritt}
- Wenn man die Beschreibung als atomaren Wert sieht, dann ist Bauanleitung in **1. Normalform**.
- 2. Normalform ist nicht erfüllt, weil $\{\text{Produkt}\} \twoheadrightarrow \{\text{Beschreibung}\}$ gilt, das Attribut Beschreibung also von einem Schlüsselattribut abhängt, der nicht der gesamte Schlüssel ist.

b)

Beispiele mit gültigen Ausprägungen:

```
('Tisch', 'Tischplatte', 1, 'massiver Eichentisch', 1)
('Tisch', 'Tischfuß', 2, 'massiver Eichentisch', 4)
('Tisch', 'Schrauben', 3, 'massiver Eichentisch', 16)
('Computer', 'Gehäuse', 1, 'Gaming PC', 1)
('Computer', 'Mainboard', 2, 'Gaming PC', 1)
('Computer', 'SSD', 3, 'Gaming PC', 2)
```

c)

Nachteil: Die Beschreibung ist direkt vom Produkt abhängig und wird trotzdem zu jedem Produkt gespeichert --> das führt zu viel Redundanz. Wenn man beispielsweise den 'massiven Eichentisch' in 'hochwertigen Eichentisch' ändern möchte, muss man das in jedem Eintrag des Tisches machen. --> Anomalien

d) Lösungsvorschlag:

Die Beschreibung kann in eine eigene Tabelle ausgelagert werden.

z.B.: Produktbeschreibungen(Bauanleitung.Produkt, Beschreibung)

So wird die Beschreibung zu jedem Produkt nur einmal gespeichert.

4) Normalformen

a) Zutrittsprotokoll

Zutrittsprotokoll(Person, Raum, Berechtigung, Zeitpunkt)

aa) funktionale Abhängigkeit

{Person, Raum} --> {Berechtigung}

ab) Normalisierung

- 1. Normalform: erreicht, keine Mengenwerte, nur atomare Werte ✓
- 2. Normalform: Es gilt {Person, Raum} --> {Berechtigung}, Berechtigung ist also abhängig von {Person, Raum}, was nicht der gesamte Schlüssel ist -- 2. NF verletzt
✗
 - Lösung: zwei Tabellen verwenden:
 - Zutrittsprotokoll(Berechtigungen.Person, Berechtigungen.Raum, Zeitpunkt)
 - Berechtigungen(Person, Raum, Berechtigung) // Berechtigungen ist die Haupttabelle, weil der Zeitpunkt null sein kann. --> null Werte vermeiden
- 3. Normalform erfüllt, weil es keine Abhängigkeiten von Nichtschlüsselattributen gibt. ✓

Also: 3.NF:

Berechtigungen(Person, Raum, Berechtigung)

Zutrittsprotokoll(Berechtigungen.Person, Berechtigungen.Raum, Zeitpunkt)

b) Mautsystem

Mautsystem(Fahrzeug&Fahrer, Maut-Durchfahrt)

1. Normalform: Stark verletzt - keine atomaren Werte verwendet.

Lösung:

- Mautsystem(FahrerID, Fahrername, Modell, Farbe, Kennzeichen, Mautstellenname, MautstellenID, Timestamp) // statt Kennzeichen könnte auch FahrerID gewählt werden. Beides existiert zum Timestamp nur einmal. FahrerID ist also auch ein Schlüsselkandidat. MautstellenID ist kein PK, weil ein Kennzeichen zu einem Timestamp auch nur an einer Mautstelle sein kann.

Dieses Schema ist in der **1. Normalform**, weil nur atomare Werte verwendet werden.

Dieses Schema ist in der **2. Normalform**, weil alle Attribute voll funktional abhängig vom gesamten Schlüssel ist.

Dieses Schema ist jedoch nicht in 3. Normalform, weil es transitive Abhängigkeiten gibt:

- {FahrerID} --> {Fahrername}
- {MautstellenID} --> {Mautstellenname}
- {Kennzeichen} definiert nichts, weil
 - Fahrer unklar
 - Auto unklar (Wechselkennzeichen)
- {FahrerID} definiert sonst auch nichts, weil der Fahrer mit verschiedenen Fahrzeugen fahren kann.

Um die **3. Normalform** zu erreichen, muss die Tabelle aufgeteilt werden.

- Mautsystem(~~Fahrer~~, FahrerID, ~~Mautstelle~~, MautstellenID, Kennzeichen, Timestamp, Modell, Farbe)
- Fahrer(FahrerID, Fahrername)
- Mautstelle(MautstellenID, Mautstellenname)

5.) Fehlersuche

a) Buch

1. Normalform verletzt: ISBN10 und ISBN13 sind in einem Attribut gemeinsam gespeichert. Diese Version erfüllt die 1. NF:

1. NF:

- Buch(ISBN13, ISBN10, Titel, Autor)

2. NF erfüllt, weil alle Attribute entweder Schlüsselattribute sind oder vom gesamten Schlüssel abhängig sind.:

- Funktionale Abhängigkeiten sind:
 - {ISBN13} --> {Titel, Autor}
 - {ISBN10} --> {Titel, Autor}
 - {ISBN13} --> {ISBN10} // und umgekehrt

3. NF

Die 3. NF ist erfüllt, weil es keine Abhängigkeiten zwischen Nichtschlüsselattributen gibt. (ISBN10 ist ein Schlüsselkandidat und kein Nichtschlüsselattribut.)

b) Automodell

Das Modell muss sich durch den Hersteller identifizieren. (Wenn z.B. die Zusammenarbeit mit einem Hersteller beendet wird, können automatisch alle Modelle entfernt, wenn man den Hersteller entfernt.)

Hersteller(HerstellerID, HerstellerName)

Automodell(ModellID, Hersteller.HerstellerID, ModellName)

Mit dieser Änderung gelten folgende Normalformen:

1. NF: erfüllt (atomare Werte, keine Listen etc.)
2. NF: erfüllt
3. NF: erfüllt (keine transitiven Abhängigkeiten)

c) Verkaufsstatistik

Verkaufsstatistik(FilialID, Filialname, ISBN, Buchtitel, VerkaufteExemplare)

1. NF: erfüllt (nur atomare Werte)
2. NF: nicht erfüllt - beispielsweise ist der Filialname von der FilialID abhängig aber nicht von der ISBN. Also muss die Tabelle aufgeteilt werden.

Filiale(FilialID, Filialname)

Buch(ISBN, Buchtitel)

Verkaufsstatistik(Filiale.FilialID, Buch.ISBN, VerkaufteExemplare)

3. NF: erfüllt - keine transitiven Abhängigkeiten

d) Rechnung

Rechnung(RechnungsNr, KundenNr, Rechnungszeile, ArtikelNr, ArtikelName, Menge)

RechnungsNr und Rechnungszeile brauchen eine identifizierende Beziehung.

Rechnungsposition(Rechnung.Rechnungsnummer, Rechnungszeile, ArtikelNr, ArtikelName, Menge)

Rechnung(RechnungsNr, KundenNr)

Das erfüllt die **1. NF** und die **2. NF** (alle Nichtschlüsselattribute sind vom gesamten Schlüssel abhängig)

3. NF: nicht erfüllt, weil {ArtikelNr} --> {ArtikelName} gilt, was eine transitive Abhängigkeit ist.

Diese Version erfüllt die 3. NF:

Artikel(ArtikelNr, ArtikelName)

Rechnungsposition(Rechnung.Rechnungsnummer, Rechnungszeile, Artikel.ArtikelNr, Menge)

Rechnung(RechnungsNr, KundenNr)

e) Arbeitszeit

1. NF: verletzt. besser:

ArbeitsEinteilung(Mitarbeiter.MitarbeiterNr, Datum, UhrzeitVon, UhrzeitBis, Arbeitsstatus)

Mitarbeiter(MitarbeiterNr, Vorname, Nachname, Abteilung)

ohne Null-Werte bei UhrzeitVon und UhrzeitBis:

UrlaubsTag(Mitarbeiter.MitarbeiterNr, Datum)

Arbeitstag(Mitarbeiter.MitarbeiterNr, Datum, UhrzeitVon, UhrzeitBis)

Mitarbeiter(MitarbeiterNr, Vorname, Nachname, Abteilung)

Ich bevorzuge die Version ohne Null-Werte. Dabei gilt:

1. NF ist dabei erfüllt.

2. NF ist auch erfüllt.

3. NF ist auch erfüllt. (keine transitiven Abhängigkeit)

f) Prüfung

Die Modellierung wie in der Tabelle macht wenig Sinn, weil die Teilnahme von der Prüfung abhängen muss. (identifizierende Beziehung)

Ein besserer Entwurf ist:

Prüfung(PrüfungsNr, Lehrveranstaltung)

Teilnahme(Prüfung.PrüfungsNr, MatrikelNr, Note, Prüfungsdatum)

Dieser erfüllt die **1. NF**, die **2. NF** (alle Nichtschlüsselattribute vom gesamten Key abhängig) und die **3. NF** (keine transitiven Abhängigkeiten).

Note: Prüfungen können an mehreren Tagen abgehalten werden - daher ist das Prüfungsdatum voll funktional abhängig von {Prüfung.PrüfungsNr, MatrikelNr}.